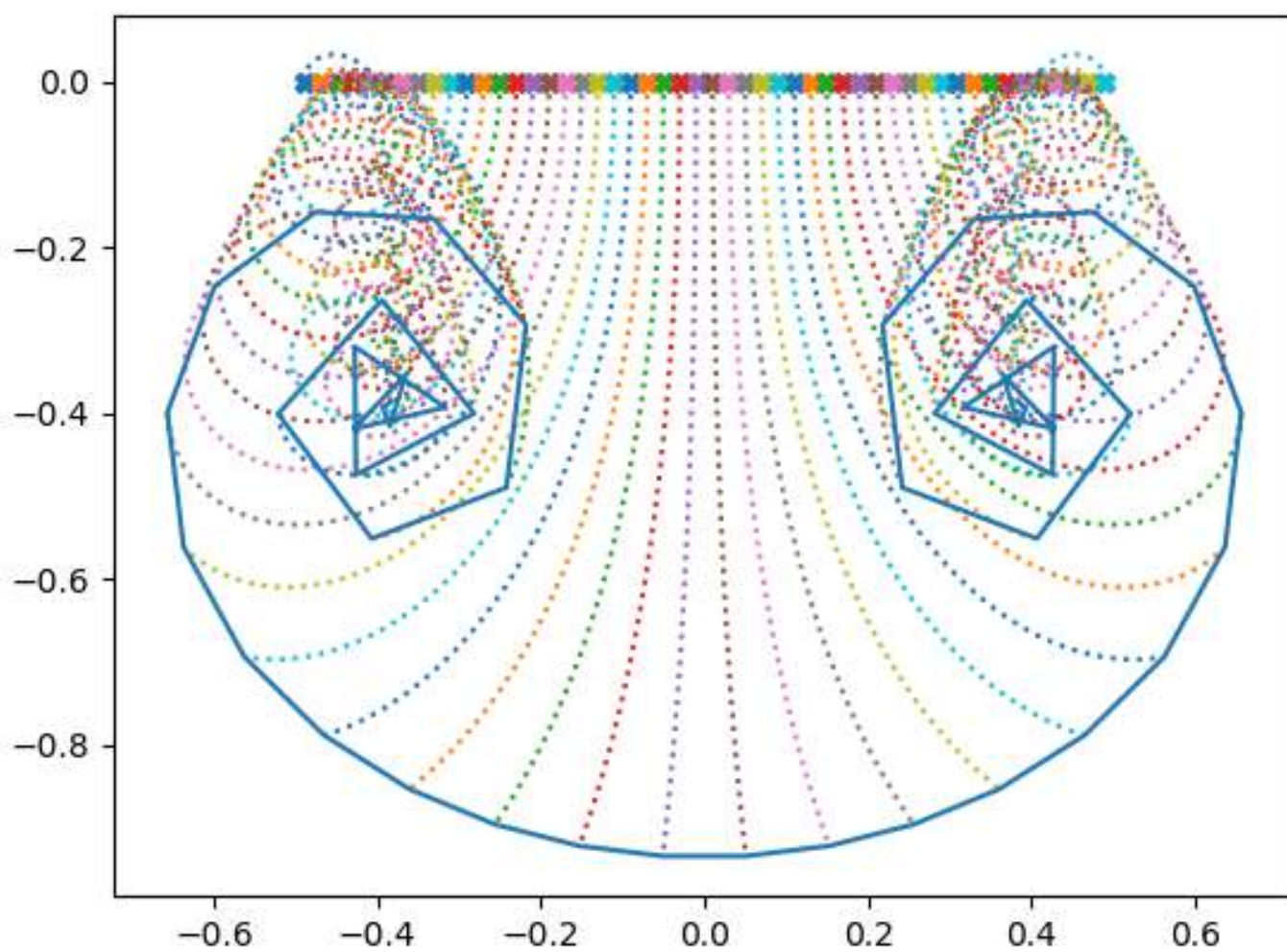
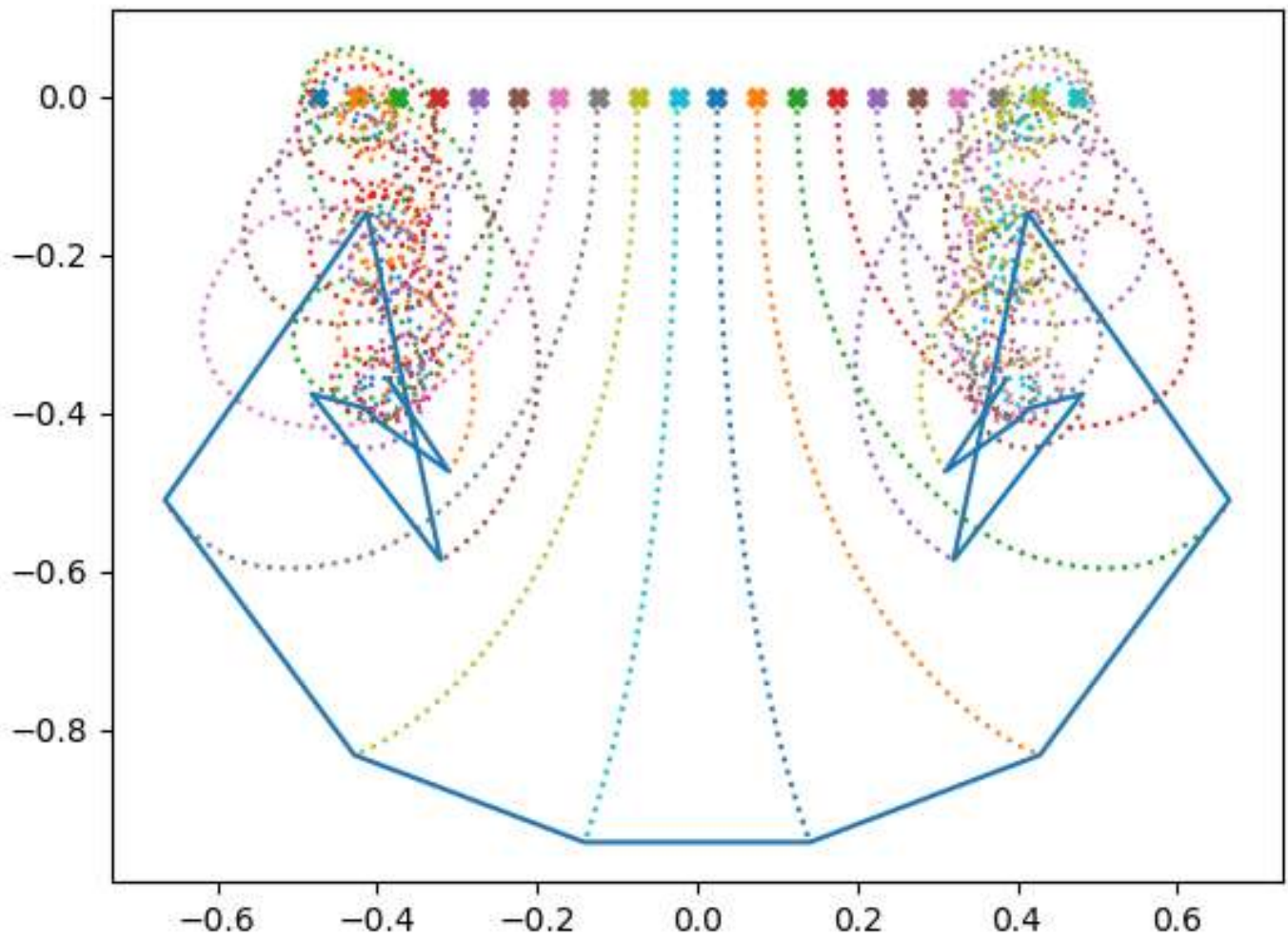


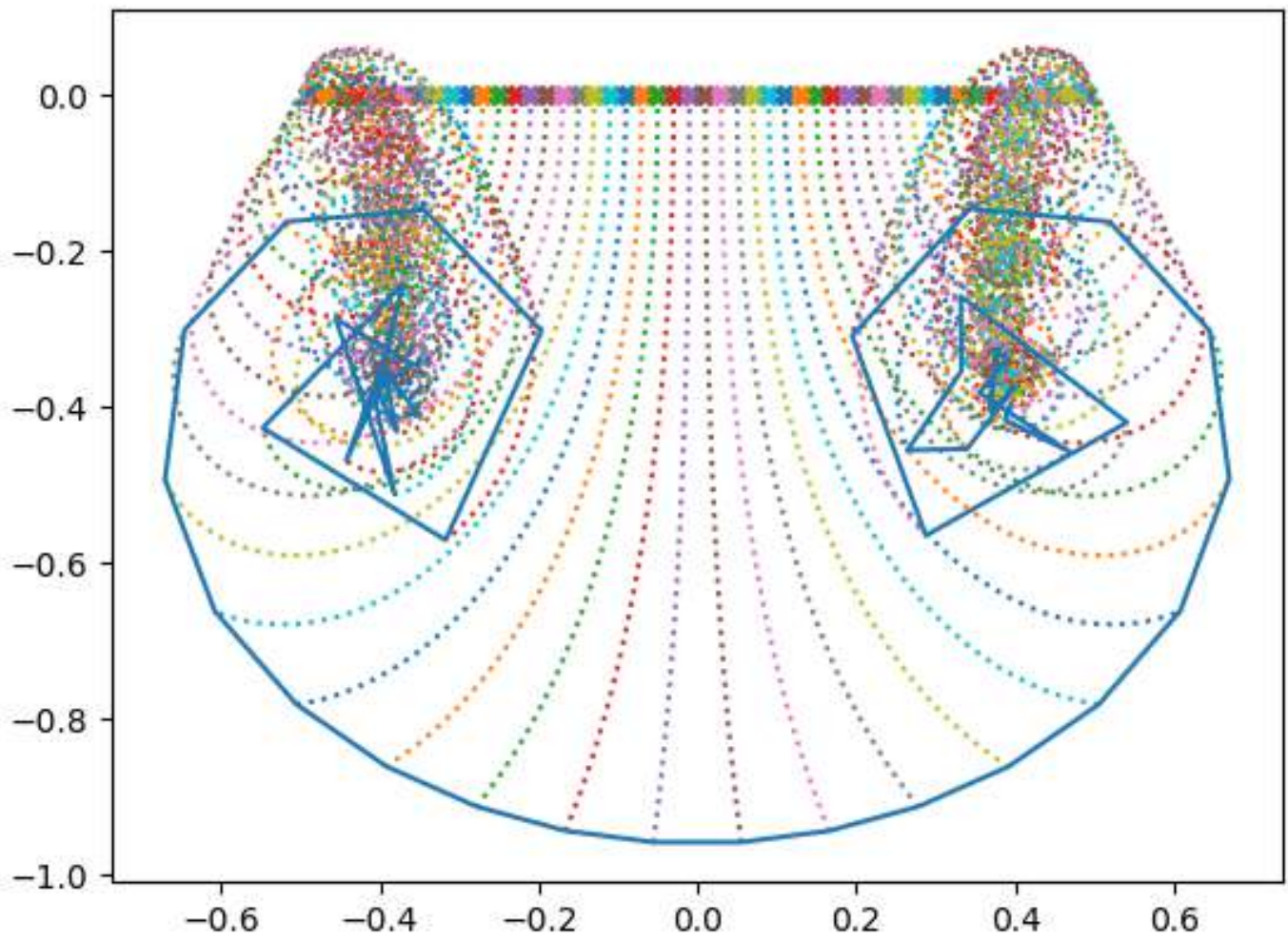
No. of particles - 50 , Time - 2s , $\delta = 4 * \delta_x$



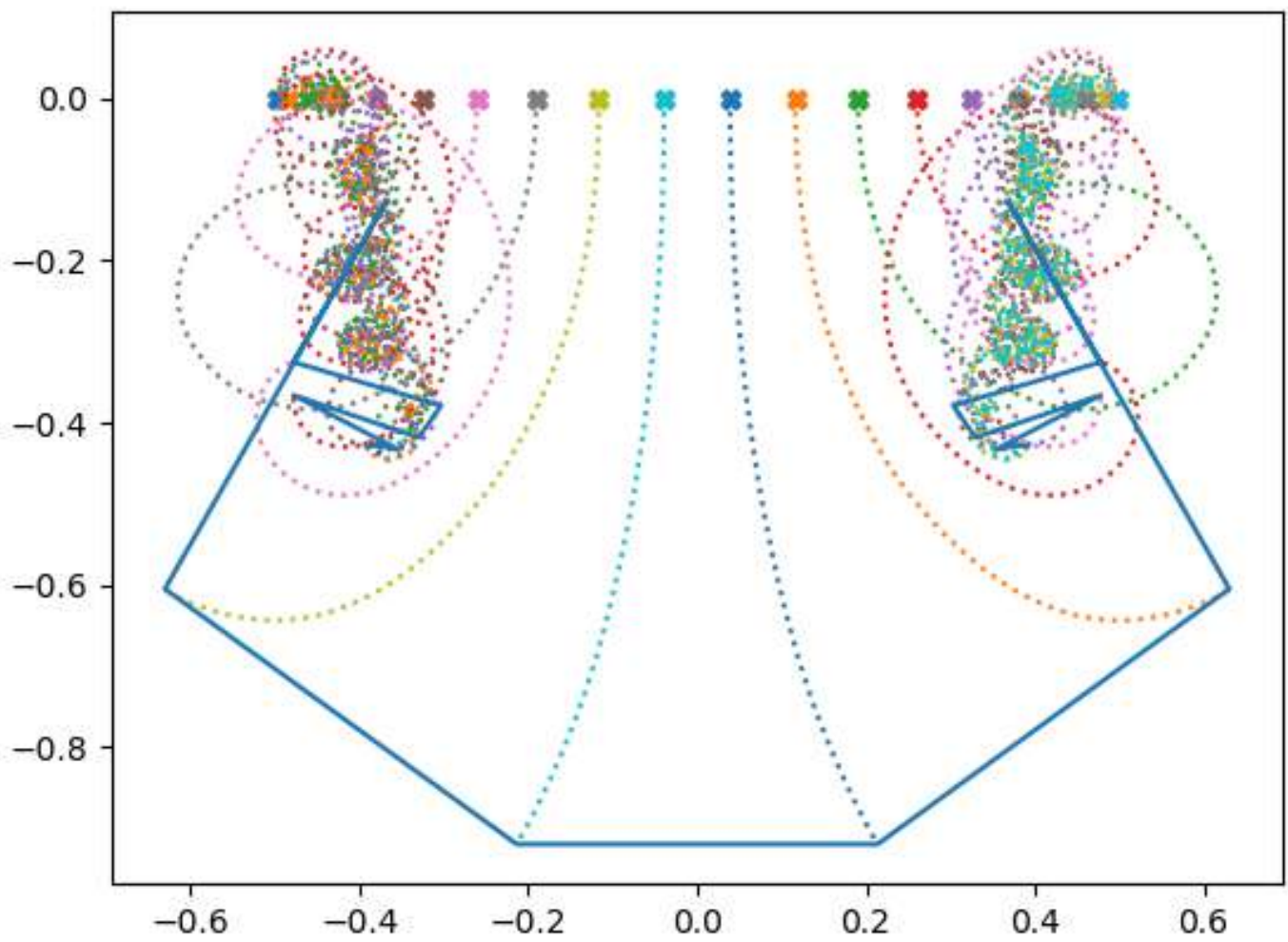
No. of particles - 20 , Time - 2s



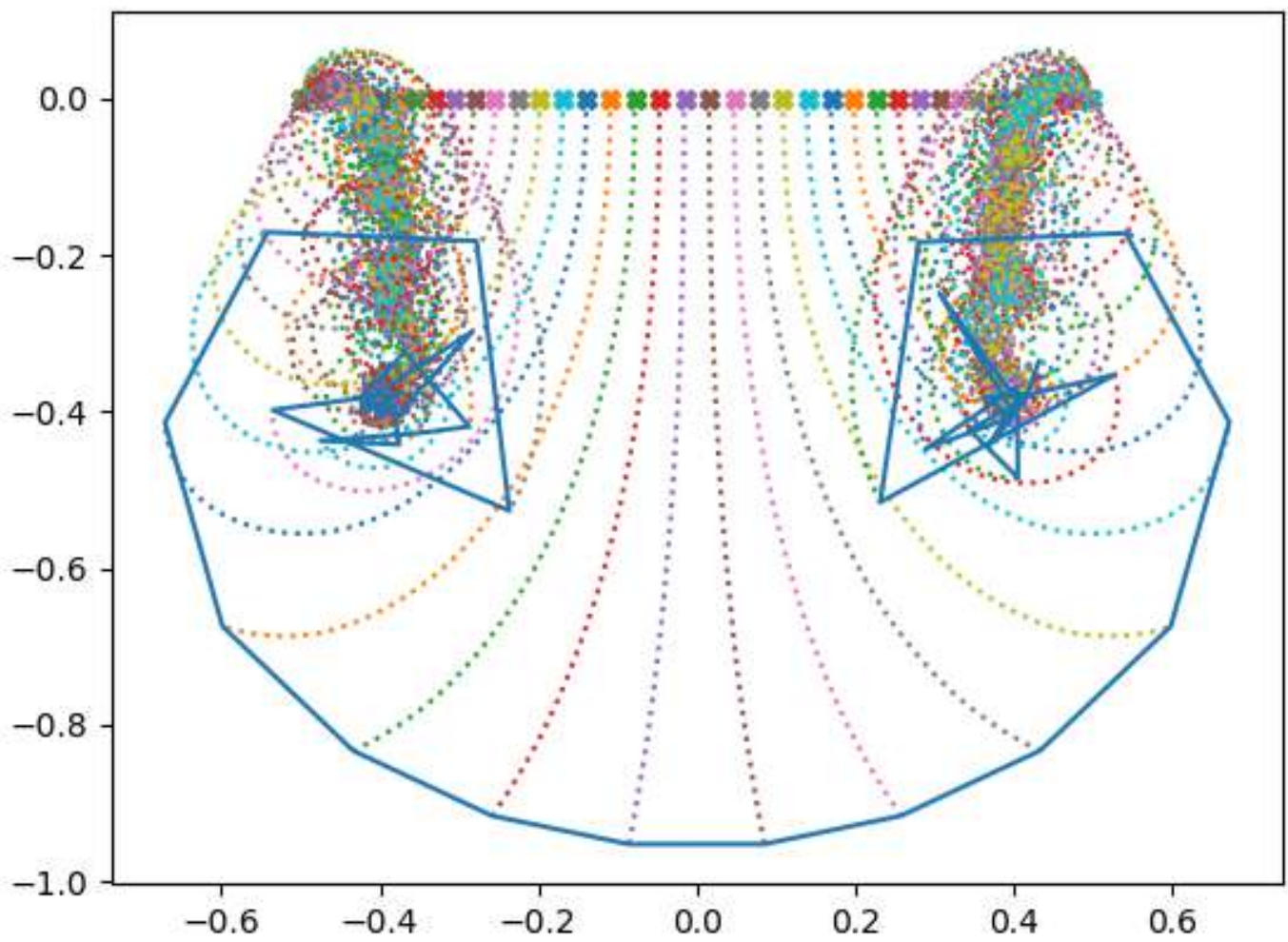
No. of particles - 50 , Time - 2s



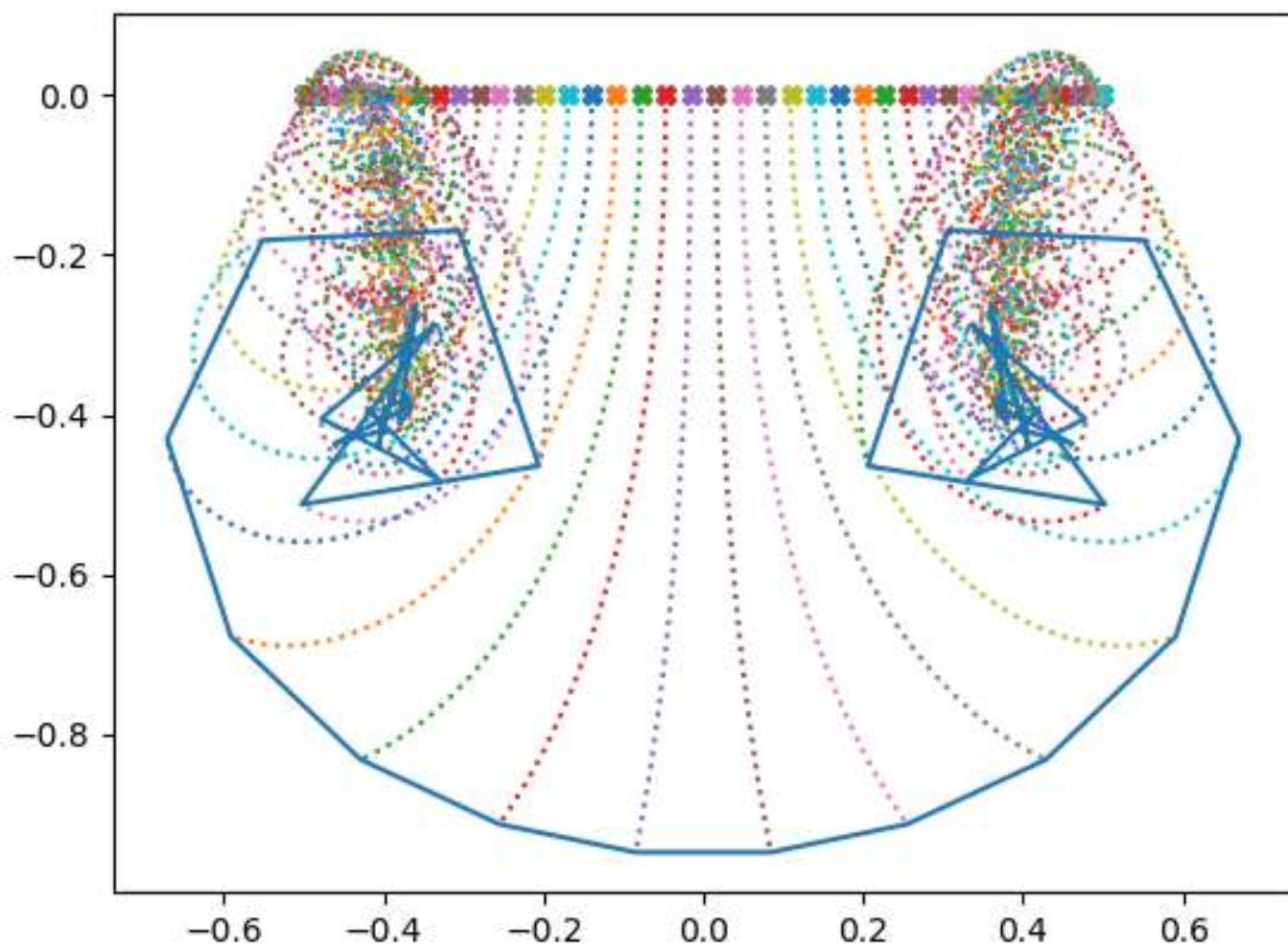
No. of particles - 20 , Time - 2s



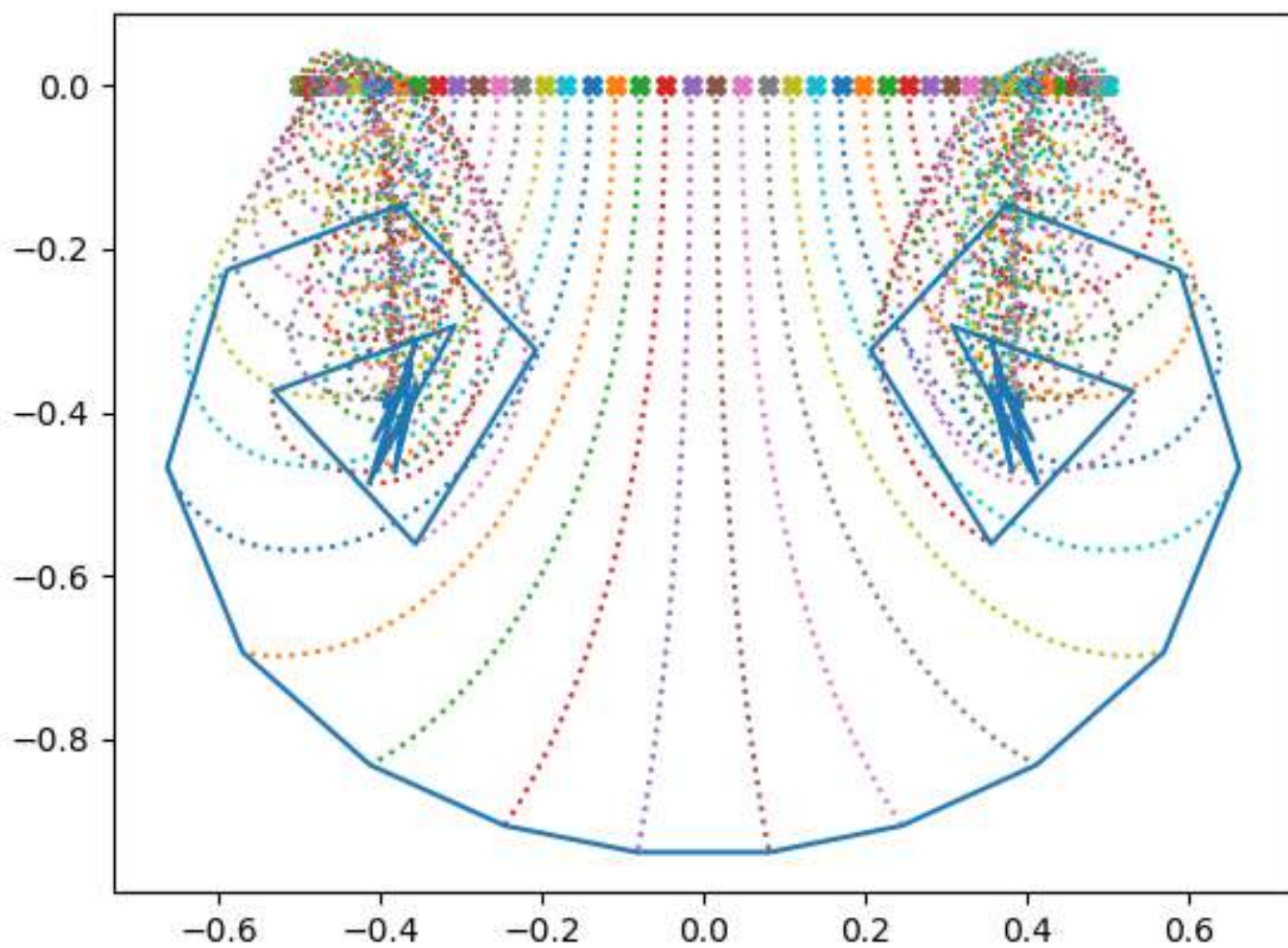
No. of particles - 50 , Time - 2s



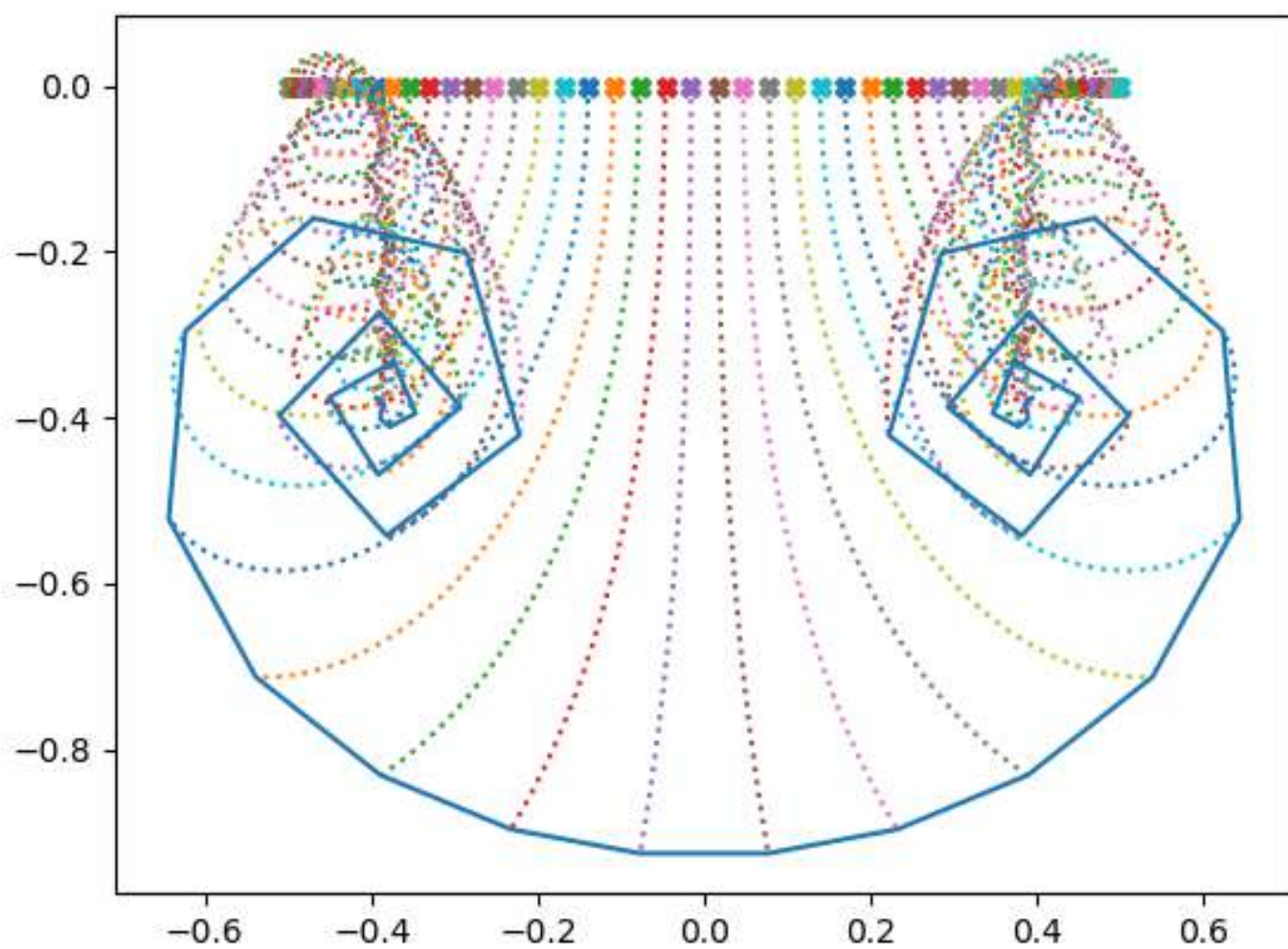
No. of particles - 50 , Time - 2s , $\delta = 1 * \delta_x$



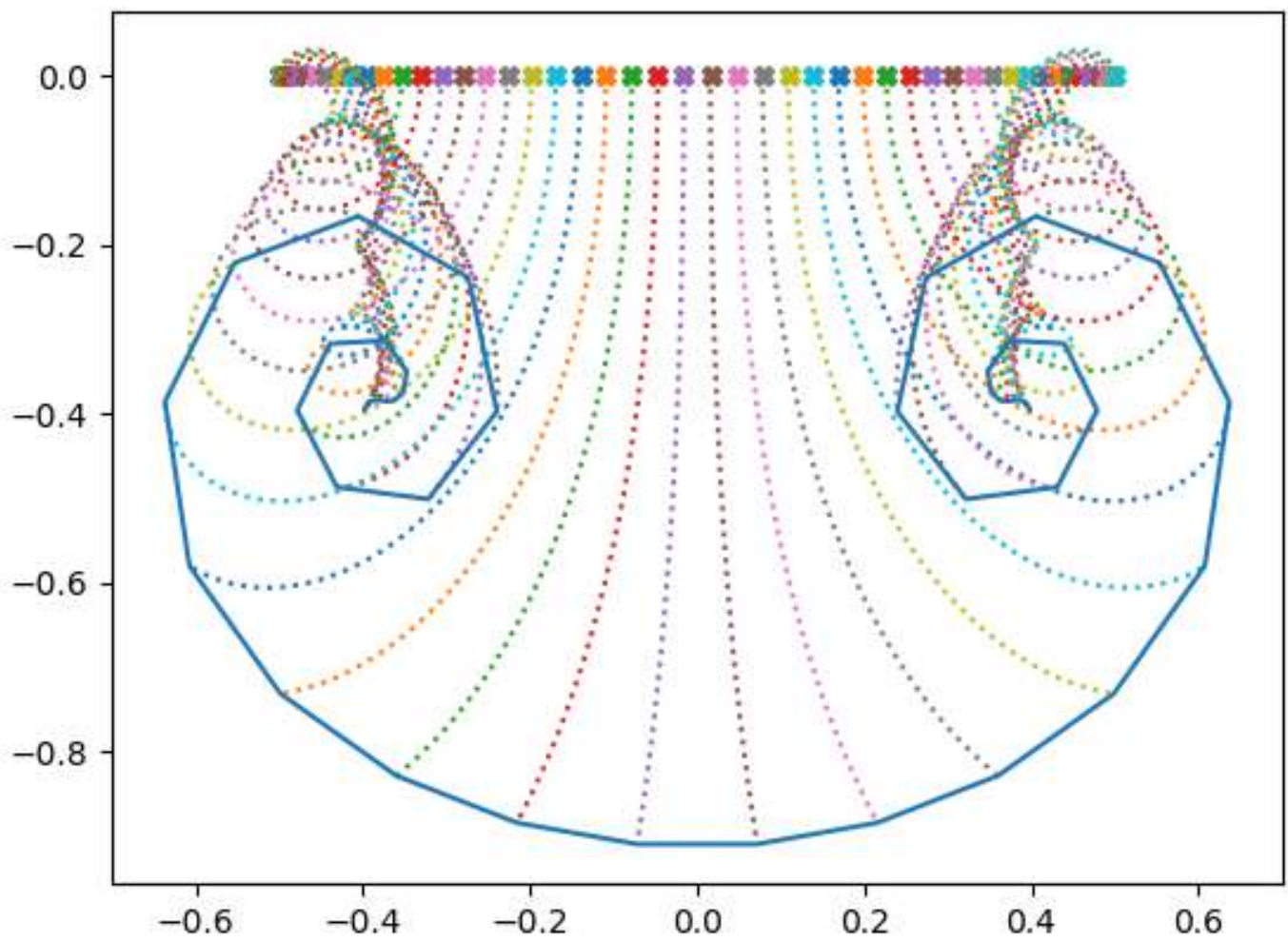
No. of particles - 50 , Time - 2s , $\delta = 2 * \delta_x$



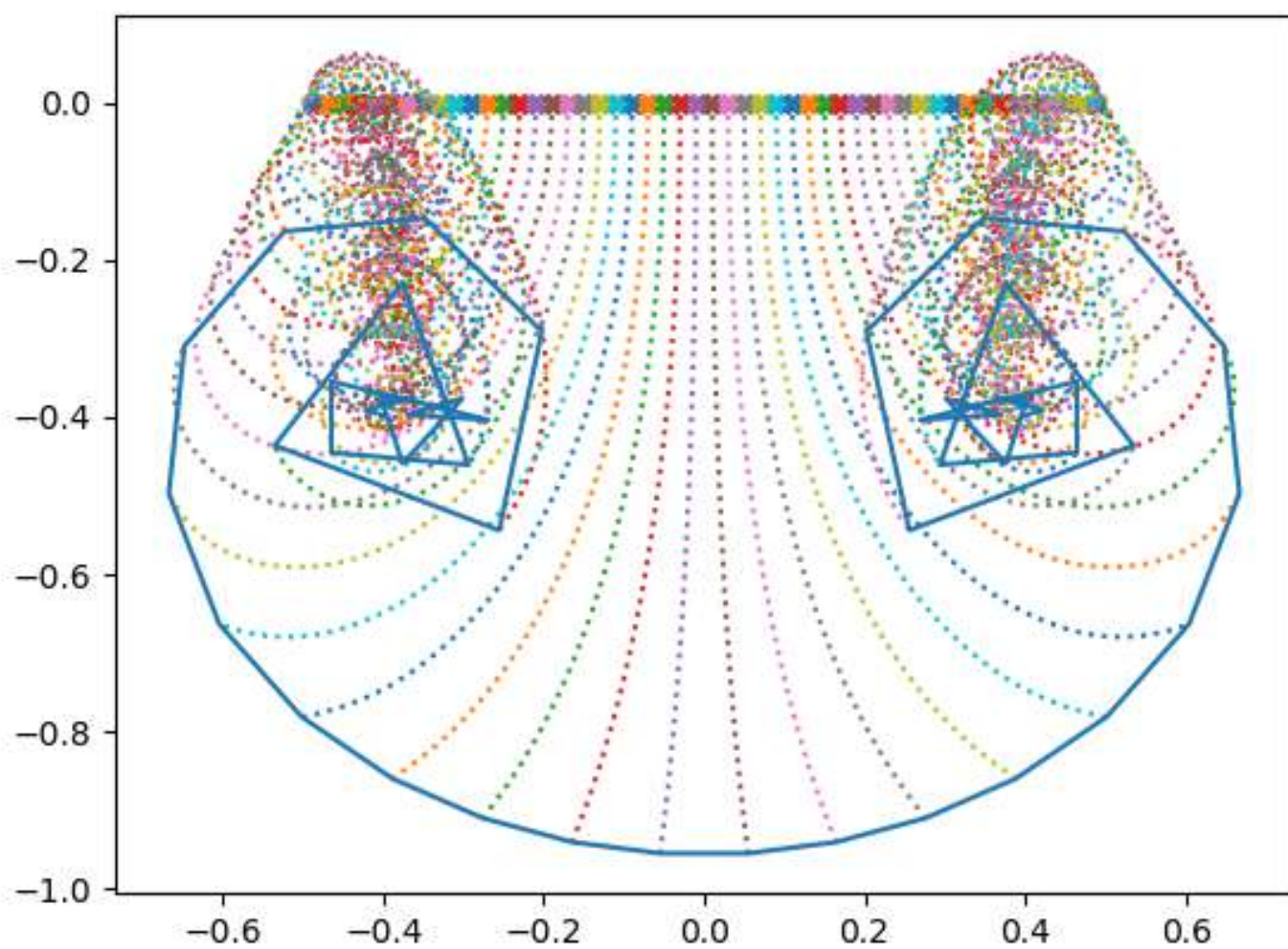
No. of particles - 50 , Time - 2s , $\delta = 3 * \delta_x$



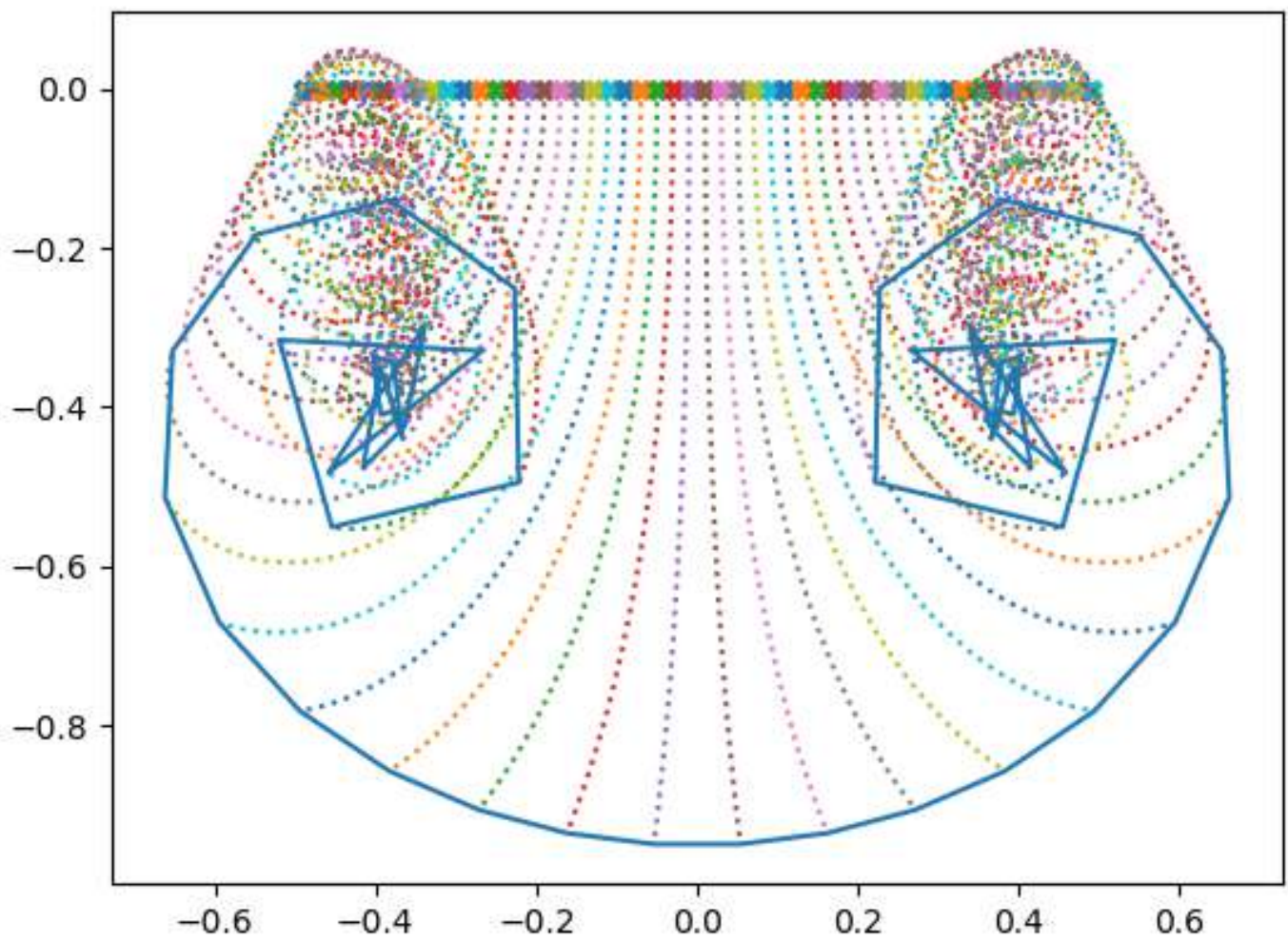
No. of particles - 50 , Time - 2s , $\delta = 4 * \delta_x$



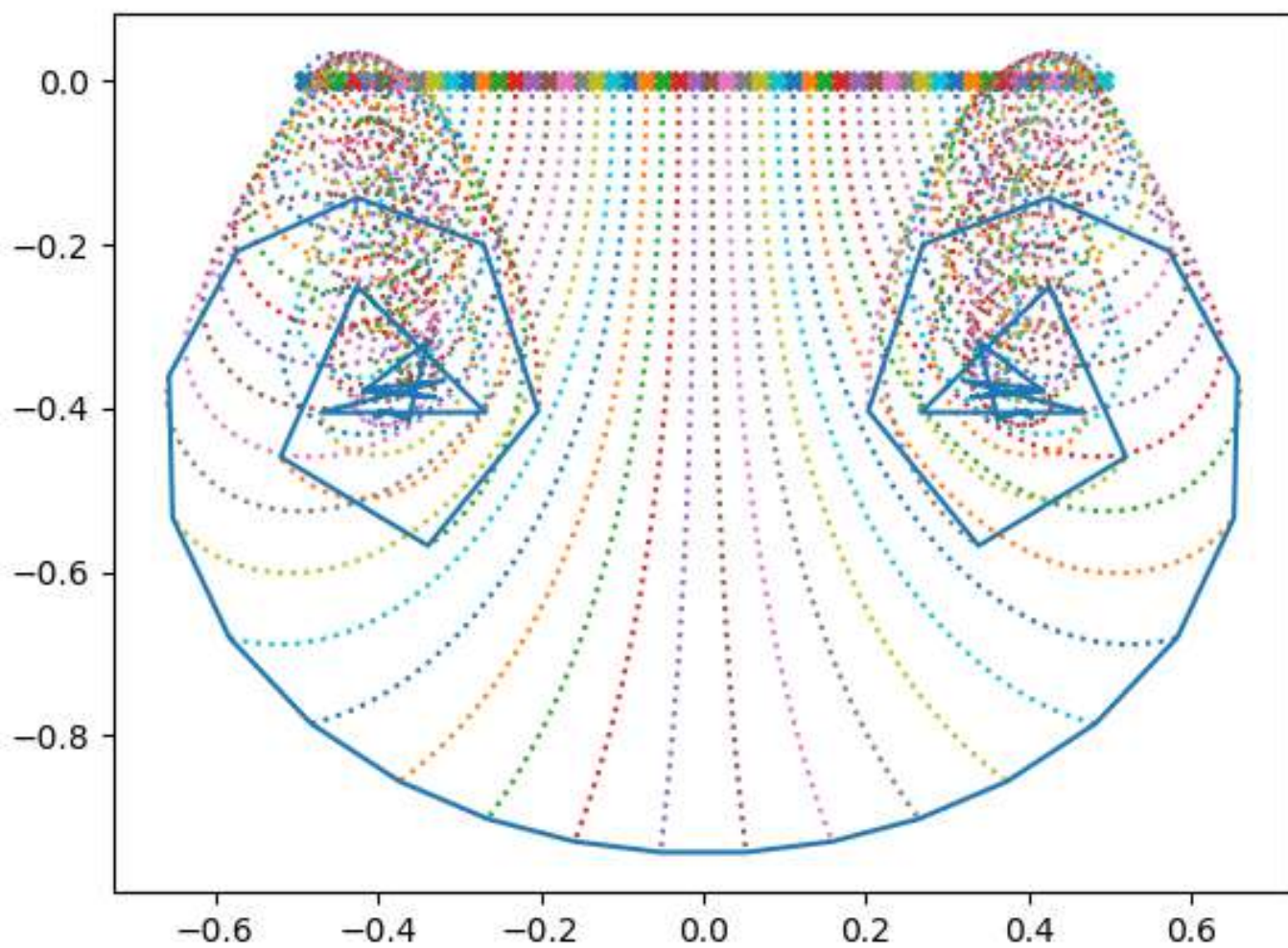
No. of particles - 50 , Time - 2s , $\delta = 1 * \delta_x$



No. of particles - 50 , Time - 2s , $\delta = 2 * \delta_x$



No. of particles - 50 , Time - 2s , $\delta = 3 * \delta_x$




```

1  import numpy as np
2  import math as mh
3  import matplotlib.pyplot as plt
4  import cmath as cm
5
6  TOS=1
7  N=200
8  del_t_x=4
9  def initial_location():
10     Pos=[]
11     for iter_1 in range(N):
12         initial_X=np.linspace(-0.5,0.5,N+1)
13         initial_X=0.5*np.sin( initial_X * np.pi / (2*0.5))
14         d_x= initial_X[1:N+1] - initial_X[:N]
15         initial_X=np.copy((initial_X[:N]+initial_X[1:N+1])/2)
16         initial_Y=np.zeros(N)
17         for i in range(int(len(initial_X))):
18             Pos[i].append(initial_X[i])
19             Pos[i].append(initial_Y[i])
20         return Pos,d_x
21
22 def Tau_i():
23     initial_X=np.linspace(-0.5,0.5,N+1)
24     initial_X=0.5*np.sin( initial_X * np.pi / (2*0.5))
25     Tau= np.sqrt(1-4*(initial_X[:N])**2) - np.sqrt(1-4*(initial_X[1:N+1])**2)
26     return Tau
27
28 def initial_vel():
29     Vel=0j*np.zeros(N)
30     return Vel
31
32 def velocity(i,Pos,Tau,d_xa):
33     z_i=complex(Pos[i][0],Pos[i][1])
34     vel_sum=0
35     for j in range(N):
36         if j!=i:
37             d_x=max(d_xa)
38             delta_sq=(del_t_x*d_x)**2
39             z_j=complex(Pos[j][0],Pos[j][1])
40             r_d=z_i-z_j
41             #vel_sum += complex(0,-0.5/(mh.pi))*Tau[j]/r_d
42             vel_sum += complex(0,0.5/(mh.pi))*Tau[j]*r_d/((abs(r_d))**2+delta_sq)
43     Vel_i=vel_sum#.conjugate()
44     return Vel_i
45
46 def Vortex_simulation(d_t):
47     Pos,d_x=initial_location()
48     Tau=Tau_i()
49     Vel=initial_vel()
50     Path_tracker=[]
51     tim=0
52     while tim<TOS:
53         # RK_2
54         Path_tracker=np.append(Path_tracker,[Pos])
55         # y(t)
56         for nop in range(N):
57             Vel[nop]=velocity(nop,Pos,Tau,d_x)
58             Pos_temp=np.copy(Pos)
59             # y tilda
60             for nop in range(N):
61                 Pos[nop][0]+=0.5*(Vel[nop]*d_t).real
62                 Pos[nop][1]+=0.5*(Vel[nop]*d_t).imag
63             for nop in range(N):
64                 Vel[nop]=velocity(nop,Pos,Tau,d_x)
65             # Y(t+1)
66             Pos=np.copy(Pos_temp)
67             for nop in range(N):
68                 Pos[nop][0]+=d_t*(Vel[nop]).real

```

```

68         Pos[nop][1]+=d_t*(Vel[nop]).imag
69         tim+=d_t
70         del Pos_temp
71         print(tim)
72     return Path_tracker
73
74
75 def Path_plotting(d_t):
76     Pos_ini,d_x=initial_location()
77     Path_track=Vortex_simulation(d_t)
78     Path_line=[None for iter_1 in range(N)]
79     loc_of_par=[[],[]]
80     for iter_1 in range(N):
81         temp_1=int(len(Path_track)/(2*N))
82         Path_line[iter_1]=[],[]
83         loc_of_par[0].append(Path_track[2*N*(temp_1-1)+2*iter_1])
84         loc_of_par[1].append(Path_track[2*N*(temp_1-1)+2*iter_1+1])
85         for i in range(temp_1):
86             Path_line[iter_1][0].append(Path_track[2*N*i+2*iter_1])
87             Path_line[iter_1][1].append(Path_track[2*N*i+2*iter_1+1])
88     plt.figure(1)
89     for iter_3 in Pos_ini:
90         plt.plot(iter_3[0],iter_3[1],'X')
91     for iter_1 in range(N):
92         plt.plot(Path_line[iter_1][0],Path_line[iter_1][1],':')
93     plt.plot(loc_of_par[0],loc_of_par[1])
94     plt.suptitle("No. of particles - "+str(N)+" , Time - "+ str(TOS)+"s"+" , delta = "+
95                 str(del_t_x)+" * delta_x")
96     plt.savefig("final.png")
97     plt.show()
98

```